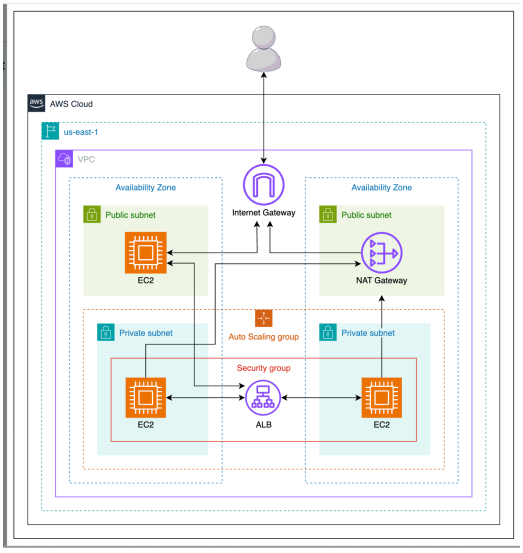
Cloud Lab - ELB - Excercise

Sunday, 21 June 2026 5:14 PM

Create and Configure ALB to Build a Resilient Application

Deploy a Resilient Web Application with Application Load Balancer

A growing online retailer wants to build a robust platform that can handle increasing user traffic without sacrificing performance or security. The business requires a solution that automatically scales and distributes user traffic across multiple backend servers deployed within secure, private networks across several data centers. The goal is to ensure uninterrupted service even if one data center fails, while enforcing strict access controls.



1. Network configuration

Set up a fault-tolerant network by creating a custom VPC with public and private subnets across multiple AZs, allowing secure internet access for private instances through a NAT Gateway.

- Create a private network named `clab-vpc`. It must use the IP range `10.0.0.0/16` and contain two private and public segments distributed across different AZs of the `us-east-1` region to ensure fault tolerance and resilience.
- Associate the private subnets with a route table directing `0.0.0.0/0` traffic to the NAT Gateway.

2. Instance-level security controls

Define security groups to control access by allowing only necessary inbound traffic for your application's frontend and backend services.

- Create a frontend security group named `clab-frontend-sg` and allow inbound traffic for HTTP, TCP (`3000`), and SSH from `0.0.0.0/0`.
- Create a backend security group named `clab-backend-sg` with inbound rules for HTTP and TCP (`3000`) from `0.0.0.0/0`.

3. Setting up the target group for incoming traffic distribution

To distribute incoming traffic across healthy backend instances, create a target group named `clab-tg`. Configure this target group to use the HTTP protocol on port `3000` within the specified VPC, and set up health checks on the same port to monitor the availability of the registered instances.

4. Incoming traffic distribution

Configure the Application Load Balancer (ALB) to distribute incoming traffic across the healthy backend target group, ensuring high availability and automatic failover.

- Create an ALB named `clab-alb` to serve internal traffic in private subnets across two Availability Zones, and associate it with the `clab-tg` target group.
- Ensure you attached an appropriate security group to the ALB that allows inbound TCP traffic on port `3000`.

5. Backend resource management

To ensure high availability of your application, deploy the backend instances using a launch template and attach them to the Auto Scaling group to dynamically allocate resources.

- Deploy and configure a group of dynamically managed backend server instances with the template name `clab-backend-template`. Use a Linux-type image with an instance type `t2.micro` or `t3.micro`, and set the storage size to 8 GiB for all EC2 instances.

- Ensure that the launch template includes the correct security group.

```
#!/bin/bash

# Update the system
yum update -y

# Install Python 3 and pip
yum install -y python3

# Upgrade pip
yum install -y python3-pip

# Install Flask and requests
pip3 install flask requests

# Create the Flask app
cat <<EOF > server.py

from flask import Flask, render_template_string
import requests

app = Flask(__name__)

def get_ec2_metadata():
    try:
        # Fetch token using IMDSv2
        token_url = "http://169.254.169.254/latest/api/token"
        token_headers = {"X-aws-ec2-metadata-token-ttl-seconds": "21600"}
        token_response = requests.put(token_url, headers=token_headers)
        token_response.raise_for_status()
        token = token_response.text

        # Fetch instance identity document for region
        identity_url = "http://169.254.169.254/latest/dynamic/instance-identity/document"
        identity_headers = {"X-aws-ec2-metadata-token": token}
        identity_response = requests.get(identity_url, headers=identity_headers)
        identity_response.raise_for_status()
        identity_data = identity_response.json()
        region = identity_data.get("region", "Unknown")

        # Fetch instance hostname
        hostname_url = "http://169.254.169.254/latest/meta-data/hostname"
        hostname_response = requests.get(hostname_url, headers=identity_headers)
        hostname_response.raise_for_status()
        hostname = hostname_response.text

        return region, hostname
    except Exception:
        return "Unknown", "Unknown"

@app.route("/")
def index():
    region, hostname = get_ec2_metadata()
    html_template = """
    <!DOCTYPE html>
    <html>
    <head>
        <title>{{ region }}</title>
        <style>
            body {
                background-color: #f8f9fa;
                font-family: Arial, sans-serif;
                text-align: center;
                padding: 20px;
            }
            h1, h2 {
                color: #333;
            }
        </style>
    </head>
    <body>
        <h1>Region: {{ region }}</h1>
        <h2>Hostname: {{ hostname }}</h2>
        <h2>Application is working</h2>
    </body>
    </html>
    """
    return render_template_string(html_template, region=region, hostname=hostname)

if __name__ == "__main__":
    # Listen on all interfaces on port 3000
    app.run(host="0.0.0.0", port=3000)
EOF

# Run the Flask app in the background
python3 server.py
```

6. Enable scalability for backend instances

Create an Auto Scaling group named `clab-app-asg` using the `clab-backend-template`.

- Configure the Auto Scaling group to maintain exactly two running EC2 instances at all times to ensure high availability and scalability.
- Attach the Auto Scaling group to the existing `clab-alb`, ensuring that the instances launched by the group are automatically registered with the `clab-tg` target group.
- Ensure that the Auto Scaling group includes the correct VPC and subnets.

7. Launch and configure the application frontend

Launch an EC2 instance named `clab-app` and configure the application to route incoming client requests through the ALB using its DNS name.

- Use a Linux-type image with an instance type of `t2.micro` or `t3.micro`. The storage size of the instance is limited to 8 GiB.
- Ensure that the instance is launched in the correct VPC and subnet with the appropriate security group.

- Ensure that the instance is launched in the correct VPC and subnet with the appropriate security group.

```
#!/bin/bash

# Update the system
yum update -y

# Install Python 3 and pip
yum install -y python3

# Install pip
yum install -y python3-pip

# Install Flask and requests
pip3 install flask requests

# Create the Flask app
cat << EOF > app.py

from flask import Flask, Response, request
import requests

app = Flask(__name__)

# URL of your backend server
BACKEND_URL = "http://<REPLACE_WITH_ALB_DNS>:3000"

# Proxy all routes to the backend
@app.route('/', defaults={'path': ''})
@app.route('/<path:path>')
def proxy(path):
    target_url = f"{BACKEND_URL}/{path}"
    try:
        backend_resp = requests.get(target_url, params=request.args)
        return Response(backend_resp.content,
                        status=backend_resp.status_code,
                        content_type=backend_resp.headers.get('Content-Type', 'text/html'))
    except requests.RequestException as e:
        return f"Error connecting to backend: {e}", 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=80)
EOF

# Run the Flask app in the background
python3 app.py
```

Open a new browser tab to view the deployed frontend application and paste `http://<FRONTEND_INSTANCE_PUBLIC_IP>:80` into the address bar.

Note: Replace `<FRONTEND_INSTANCE_PUBLIC_IP>` with the instance's public IP.

Great effort completing this Challenge Cloud Lab! If you found any part of this difficult, we recommend exploring our guided labs to strengthen your skills before retrying this challenge.